

Evaluation of deep learning approaches for oil & gas pipeline leak detection using wireless sensor networks

Christos Spandonidis, Panayiotis Theodoropoulos^{1,*}, Fotis Giannopoulos, Nektarios Galiatsatos, Areti Petsa

Prisma Electronics, Research, and Development, Greece¹

ARTICLE INFO

Keywords:

Leakage detection
Oil pipeline
Deep learning
CNN classifier
LSTM autoencoders

ABSTRACT

Pipelines are one of the most common systems for storing and transporting petroleum products, both liquid and gaseous. Despite the durable structures, leakages can occur for many reasons, causing environmental disasters, energy waste, and, in some cases, human losses. The object of the ESTHISIS project is the development of a low-cost and low-energy wireless sensor system for the immediate detection of leaks in metallic piping systems for the transport of liquid and gaseous petroleum products in a noisy industrial environment. In this study, two distinct leakage detection methodologies are presented. First, a 2D-Convolutional Neural Network (CNN) model undertakes supervised classification in spectrograms extracted by the signals acquired by the accelerometers mounted on the pipeline wall. This approach allows us to supplant large-signal datasets with a more memory-efficient alternative to storing static images. The second methodology entails a Long Short-Term Memory Autoencoder (LSTM AE), which directly receives the signals from the accelerometers, providing an unsupervised leakage detection solution. Field tests for the validation of our methods were performed using an experimental pipeline network, while evaluation of their efficiency in a real environment was conducted in the premises of an oil refinery in Greece. Results evince the potency of the LSTM AE to recognize in real-time the emergence of deficiencies and the efficacy of the CNN models to classify accurately spectrograms reflecting the operational condition of the monitored pipelines.

1. Introduction

For thousands of years, pipelines have been constructed in various parts of the world to convey water for drinking and irrigation. The development of high-strength steel pipes made it possible to transport natural gas and oil over long distances (Liu, 1998), rendering them among the safest and most economical methods for transporting and storing oil and gas products (Callan, 2008) and a key driver for worldwide economic growth (Li et al., 2013). Oil piping installations worldwide have been rapidly expanding to satisfy the ever-increasing energy needs of the population, intruding the topological complexity of the pipeline network, perplexing its supervision, and assessment of its safety (Wang et al., 2022). This breadth of pipeline usage inherently aggrandizes the probability of structural defects due to erosion over time, fracture propagation, human factors, environmental factors, and other causes (Adegboye et al., 2019; Aryai et al., 2022; Gargade and Ohol, 2021). Among the most commonly observed types of failures is pipelines leak. Accidents like the one in Alaska's North Slope area (2006), where a spill of about 1 million liters of oil was caused by a quarter-inch hole corroded in a pipeline (Rehman and Nawaz,

2017), approve that even seemingly small defects scale up fast and avert excessive financial losses and, most importantly, eliminate any potential threat to human lives and the environment.

Development of a mathematical framework to analyze the reliability of the piping network, based on conducting failure analysis of the critical components of the network and providing insight into the pipeline network safety (Wang et al., 2022; Xie and Tian, 2018) has been proved to be a painstaking task. For this reason, an inspection of the pipe network's functionality ensures safe operation and should be an indispensable element of their functioning. Monitoring the pipe system state and the capability for on-time detection of probable leakages is a cost-effective technique facilitating the maintenance of the piping network and most importantly, contributes to the abatement of accidents (Khan et al., 2021). Several studies have been conducted to monitor dynamic systems for potential defects by analyzing their vibration response characteristics, using digital signal processing techniques, such as fast Fourier transform (Shibata et al., 2009), and the wavelet transforms (Mpesha et al., 2002). Several proposed methods for pipe inspection include the use of ground penetration radar (Hoarau et al., 2017; Ni

* Corresponding author.

E-mail address: panagiotis.theodoropoulos@prismael.com (P. Theodoropoulos).

¹ rdprojects@prismael.com

et al., 2010), infrared thermography (Manekiya and Arulmozhivarman, 2016), piezoelectric materials such as Lead Zirconate Titanate (PZT) (Siswanto et al., 2019), acoustic emission (Mahmutoglu and Turk, 2018; Martini et al., 2016), accelerometers mounted on the exterior walls of pipelines (Yazdekhesti et al., 2017) and fiber-optic sensors (Jia et al., 2018; Png et al., 2018). The main drawback of these studies is the necessity for field testing and/or the need for computationally costly data post-processing analysis.

Within the context of the 4th Industrial revolution, efficient implementation of deep learning techniques in different engineering research and industrial applications became feasible. Deep Learning (DL) has been particularly prominent for failure detection in numerous engineering disciplines, such as aerospace (Lu et al., 2020; Zhang et al., 2021), civil (Abdeljaber et al., 2018, 2017; Lin et al., 2017), mechanical (Lu et al., 2017; Weimer et al., 2016), and marine (Theodoropoulos et al., 2021) engineering, where their implementation entails several advantages such as avoidance of hand-engineered feature extraction (Adegboye et al., 2019) and system scalability among others. Similarly, several studies have proposed a variation of ML–DL configuration to perform leakage detection in pipeline systems, instead of traditional threshold-based techniques, intending to leverage the efficacy and the increased accuracy granted by neural networks in identifying even relatively small leakage diameters (Li et al., 2019). In this regard, FFNNs have been successfully utilized as a foundation for anomaly detection in piping networks processing data either in the time domain (Amini et al., 2020; Shrivani et al., 2019) or in the frequency domain (Liao et al., 2021). Even with the most recent advancement in hardware and tensor multiplication dedicated GPUs, the tuning of the hyperparameters and the training of the models is the most cumbersome part. For this matter, utilization of pre-trained models has also been widely suggested, yielding very promising results outperforming in many cases use-case-specific models (Shukla and Piratla, 2020; Zhou et al., 2021).

Additionally, autoencoders are also a class of neural networks whose attribute of being trained on unlabeled data and distinguishing potential digression from the nominal state becomes very beneficial for the detection of faulty conditions in the pipelines (Cody et al., 2020; Wang et al., 2020). Lastly, Convolutional Neural Networks (CNN) are utilized to perform feature extraction and learn through a series of filters to identify salient features in the data. These features are received by Multi-Layer Perceptrons (MLPs) (Chen et al., 2018; Hu et al., 2021b; Shi et al., 2019), or Support Vector Machines (SVM) (Kang et al., 2018) to conduct the classification, and determine the operational state reflected by the initial signal. Although, feature extraction is integral for classification methods achieving high leak detection accuracy (Hu et al., 2021a), the main limitation of all these methods is their necessity for high computational cost since post-processing analysis is performed on historical data in the cloud, additionally to the need for storage of high volume of data which makes both training and execution time of the models inefficient.

To address these issues, the “ESTHISIS” project (Nikolaidis et al., 2019) aims to detect leakages in oil and gas pipelines by integrating the best possible technologies from wireless sensor networking and edge computing with novel Situational Awareness techniques and enhance the operational capacity of oil and gas decision-makers. In our previous work (Christos et al., 2021a,b) all critical hardware and software details of the acquisition system were presented. The focal point of the current work is to describe two distinct DL-based methodologies for leakage detection in oil pipelines capable to be performed, after the training stage at the edge. The first method entails a supervised approach based on the transformation of the data to the time–frequency domain and creating spectrograms from the acquired sensors data and the use of a 2D-Convolution Neural Network to characterize the pipeline they are coming from as healthy or not. The second method is an unsupervised approach employing Long Short-Term Memory autoencoders (LSTM AE) trained to reconstruct signals from healthy pipelines alone. The model design was based on a similar approach we performed for the

performance analysis of vessels (Theodoropoulos et al., 2022), which was redesigned to include spectrogram images following (Altabay et al., 2021; Bianchi et al., 2020; MacDonald et al., 2020; Xie et al., 2020). Emphasis has been placed on the validation (field test in large-scale experimental pipeline network) and verification (pilot testing in oil refinery premises) of the methodology. Parametric tests were performed for the evaluation of model accuracy and efficiency.

Motivation of our work is the development of a methodology capable to integrate a hybrid data fusion architecture in real oil transferring pipeline infrastructure in real-time, running DL applications in a scalable manner for real-time analytics, and providing accurate estimations for leakage detection. Our modeling approaches present the benefit that particularly long time series can be converted into relatively low-resolution static images, which is a more memory-efficient solution (CNN) as well as not requiring a plethora of samples appertaining to both inspected states (i.e., healthy, defective) available during the training phase since these models trained solely on healthy signals (LSTM AE). To the best knowledge of the authors, there is no similar solution capable to operate in such a harsh environment.

The rest of the paper is structured as follows: In Section 2, a description of the processing system responsible for the data acquisition is provided. In Section 3, the implemented methodology serving as the foundation of the leakage detection scheme is delineated. In Section 4, the results from the experimental field-testing phase are shown while in Section 5, the pilot testing of our methodology in actual operating conditions is presented. Results from the comparison of the proposed method with other methodologies found in the literature are provided in Section 6. Lastly, concluding remarks and future steps are discussed in Section 6.

2. Data acquisition and processing system

The general architecture and data flow of the “ESTHISIS” system are shown in Fig. 1. The main pillars of the system architecture are the smart nodes, which are placed along with a pipeline network, 100 m–130 m apart from each other. Each node collects vibration data from the pipe walls via low-frequency accelerometers and processes the acquired data at the edge. However, two consecutive nodes can potentially function as a pair to precisely determine the characteristics of a leakage, like its exact place on the pipe. In Fig. 1, a simplified depiction of the system with two smart nodes is presented.

Fig. 1(a) presents the nodes in what is called “leak detection mode”, during which the two nodes collect measurements from the sensors and process them individually to identify a potential leakage. Fig. 1(b) represents the “leak localization” mode. In case of detecting an abnormality, the primary node activates the leak localization mode and informs the secondary node to enter this operational mode. The nodes start collecting data synchronously, with the common timing reference provided by the PPS signal of the nodes’ GPS Units. Upon leakage detection, the primary node transmits a JSON message with information regarding the leakage, like its exact location, to a Central Unit via a Narrow Band IoT connection. It is noted that within the context of the current work only the leak-detection mode is used. The Leak-localization mode involves different ML methods that will be described in our next work. The architecture of each node is presented in Fig. 2.

As shown each node comprises two major sub-systems: the Sensor Data Receiving Interface and the Data Processing Interface. Other parts of the architecture are the sensor used, the power supply, and the energy harvesting system. The design of both the hardware and the embedded software is based on PrismaSense™ technology (Christos et al., 2020), upgraded for data acquisition from low-frequency accelerometers, and enhanced with edge computing features and functionalities. The sensor data receiving interface is designed to collect low frequencies (up to 12 KHz) to increase the range of sensors and thus reduce cost. The primary processing module included in the Data Processing Interface is the RK3399Pro System-On-Module which consists of a dual-core ARM Cortex-A72 and a quad-core ARM Cortex-A53

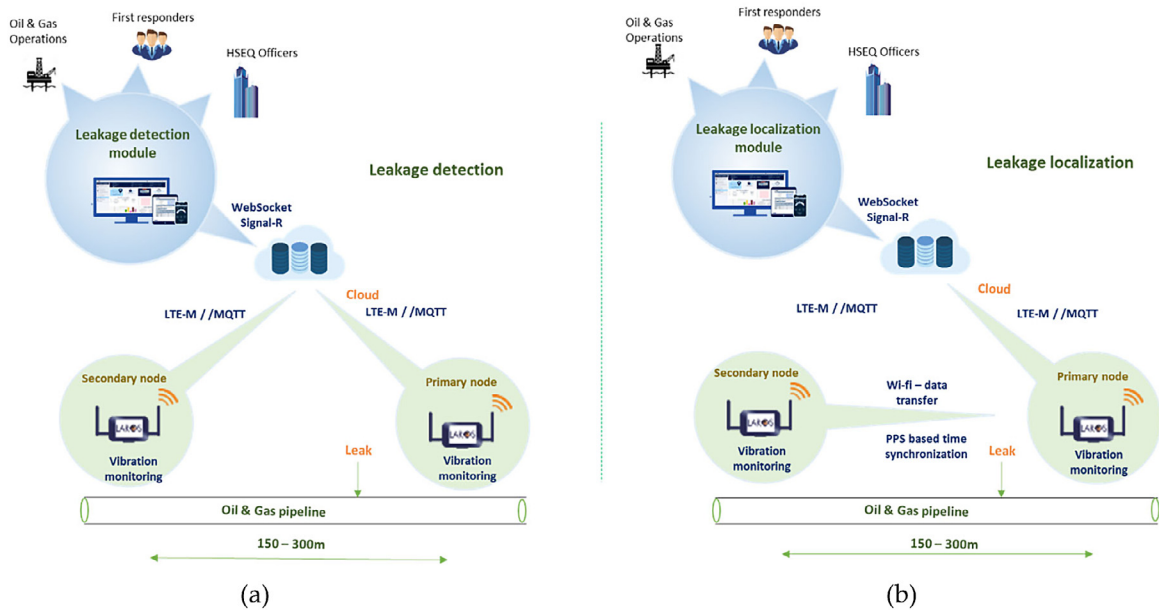


Fig. 1. ESTHSIS platform system architecture for leakage detection (a) and leakage localization (b). Systems mounted on the pipeline are depicted in green, while blue for cloud-based services.

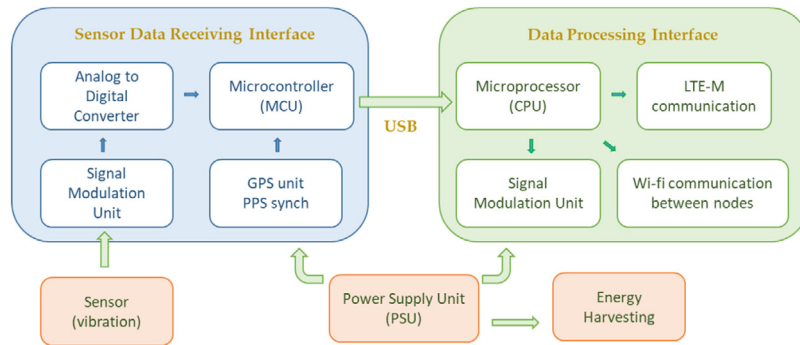


Fig. 2. Smart node design architecture.

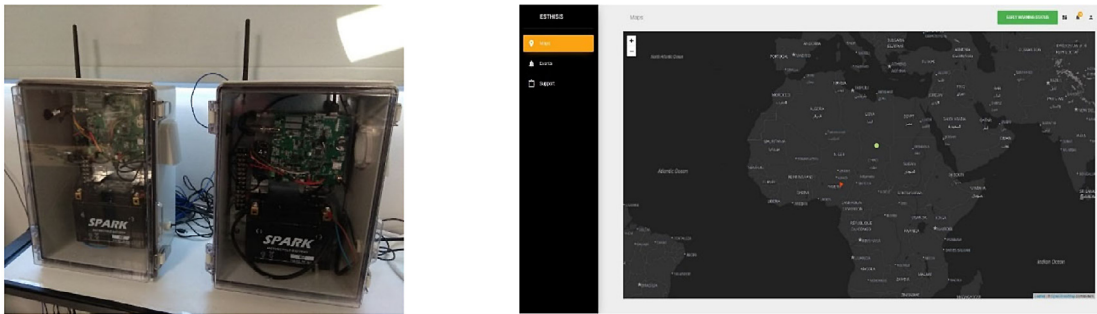


Fig. 3. Two smart nodes set up for field tests (left) and the pipeline overview web application.

microprocessors. The two nodes that were put together and used for the field tests are presented in Fig. 3 (left). The entire processing procedure is executed at this level.

In case of leakage detection, each node sends a message to the Central Unit over a Narrowband IoT (NB-IoT) connection, using the MQTT protocol, to notify the end-user about the leakage and its properties. The Central Unit, which could be either a server or a cloud infrastructure, is the means via which the end-user is informed about the leakage in the pipeline system, via a dedicated web application Fig. 3 (right).

3. Method description

Two disparate methods have been employed in pipeline network monitoring and compared based on their efficacy in detecting abnormalities and deviations from the nominal state implicating damage. The first approach concerns the utilization of the available high-frequency dataset to generate spectrograms, which are then fed to 2D-CNNs to classify spectrograms appertaining to a healthy pipeline or pipelines with leakage. A second proposed methodology is an unsupervised approach employing LSTM AE. More specifically, we train these models

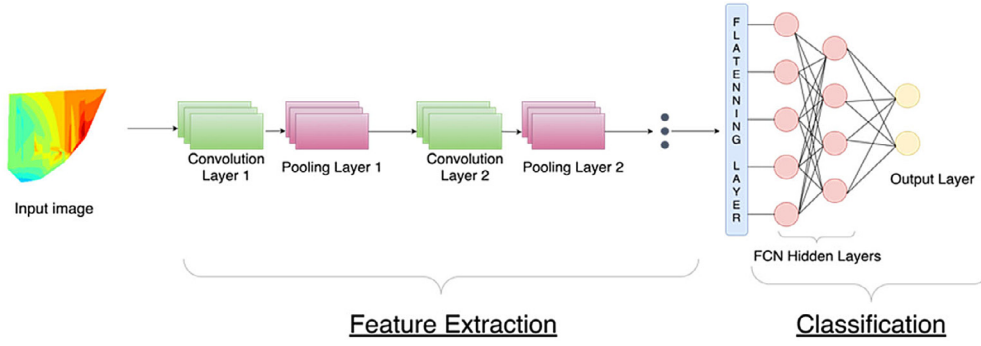


Fig. 4. Conceptualization of study case leading to the prediction of the CNN.

based only on signals corresponding to healthy pipelines to reconstruct the input data efficiently. Conversely, when facing anomalies, the model's capacity to reproduce its input deteriorates significantly.

3.1. 2D CNN for image classification

CNNs are a genre of neural networks demonstrating excellent capabilities in tasks such as pattern recognition, computer vision, and time series classification, due to their capacity to extract salient features without requiring prior domain expertise prompting researchers and developers to approach complex pattern recognition tasks. Fig. 4 demonstrates the essential steps of our methodology, which are described below in more detail.

Input:

Due to the high-frequency sampling implemented, the acquired measurements for the inspection of the piping network are rendered inefficient. The alternative of converting the signals to static images was considered and exploiting the high-frequency trait of the acquired dataset, the option of spectrograms was concluded. The spectrograms extracted by the dataset constitute the input images reflecting the operational state of the monitored pipelines. It was concluded that increasing the resolution of the input images up to a resolution of 256×256 , enabled the CNNs to be more accurate, thus this resolution was chosen for the input images. These images are converted into a 3D tensor of dimensions: Height $\times$ Width $\times$ Channels, representing the numerical value of each pixel to the components of the selected color plane (i.e., grayscale, RGB, RGBA). In our case study, the RGB color plane was chosen. Thus, the dimensions of each tensor are $256 \times 256 \times 3$.

This method offers a more memory-efficient alternative to representing lengthy signals in static images. Explicitly, a Comma Separated Values (CSV) file representing 10 s of operational data occupies 13 MB of memory, however converting the same signal into a spectrogram image of resolution (256×256) occupies less than 300 kB.

Additionally, after experimentation with multiple color maps for the generated spectrograms, it was determined that the CNN classifiers were more accurate in their inferences when the values of the one-color plane were predominant in the representation of the pixel values to the RGB plane. Therefore, a mainly blue color map was selected as depicted in Fig. 5(a).

Finally, in the context of this methodology, it is aimed to examine whether these classifiers could accomplish to classify the sample images, in case the occurrence of the rupture in the pipeline is not recorded. For this matter, we purposefully omit the time point where efflux occurs from these spectrograms to examine the efficacy of these classifiers to detect leakage, as illustrated in Fig. 5.

Feature extraction:

The operation of convolution enables these networks to learn efficiently and automatically to detect significant features in the images

without requiring any human intervention. This implies that CNNs perform independently the arduous task of feature engineering relieving engineers and researchers of that burden. Explicitly, the developed classifier is trained to identify features in the input images and determine the state of the pipeline. The operation of convolution is a linear element-wise multiplication (dot product) between a small array called the kernel of dimensions $N_a \times N_b$ gliding through the input tensor of the layer and the elements of this tensor. This process with multiple kernels allows the networks to recognize diverse patterns in the input images. Summing the fragments generated by the dot product between the filter and the corresponding portion of the input tensor it audits yields the value of the output tensor in the respective position. Convolution on the image, namely on a 2D plane, with a resolution $H \times W$ and N_c color channels take place as follows:

$$F(x, y, k) = K^o S(x, y, k) = \sum_{i=1}^{N_a} \sum_{j=1}^{N_b} \sum_{k=1}^{N_c} K(i, j, k) S(x+i-1, y+j-1, k) \quad (1)$$

Fully Connected (FC) Layer:

The conclusion of this pattern of alternating convolutional and pooling layers leads to the flattening of the final output tensor into a vector, which constitutes the input of a traditional FC Network succeeding the configuration of convolution and pooling layers. The output of a node j at the ℓ th layer of the Fully Connected Network can be expressed as follows:

$$z_j^{[\ell]} = \mathbf{W}^{[\ell]T} \cdot \mathbf{a}^{[\ell-1]} + \mathbf{b} = \sum_{k=1}^{n^{[\ell-1]}} w_{jk}^{[\ell]} a_k^{[\ell-1]} + b_j^{[\ell-1]} \quad (2)$$

$$\rightarrow a_j^{[\ell]} = f(z_j^{[\ell]}), \quad (3)$$

where $f(x) = \text{ReLU}(x) = \max(0, x)$ and $n^{[\ell-1]}$

: number of nodes at the previous layer

Classification:

Based on the feature extraction by the convolution layer, and the forward propagation of the information in the FC layers, the nodes of the last layer of the FC Network and by extension, of the whole CNN, classify the state of the piping network reflected by the sample image. The output of the last layer converts the estimations of the network into a probability distribution over the predicted classes through the SoftMax activation function, hence the summation of all outputs adds to 1.

$$a^{[L]} = \text{Softmax}(\mathbf{W}^T \cdot \mathbf{a}^{[L-1]} + \mathbf{b}) = \frac{e^{z_i^{[L]}}}{\sum_{i=1}^M e^{z_i^{[L]}}} \quad (4)$$

Loss: In neural networks, the loss function quantifies the digression between the predicted values and the ground truth labels assigned to the samples of the dataset. Therefore, the minimization of the

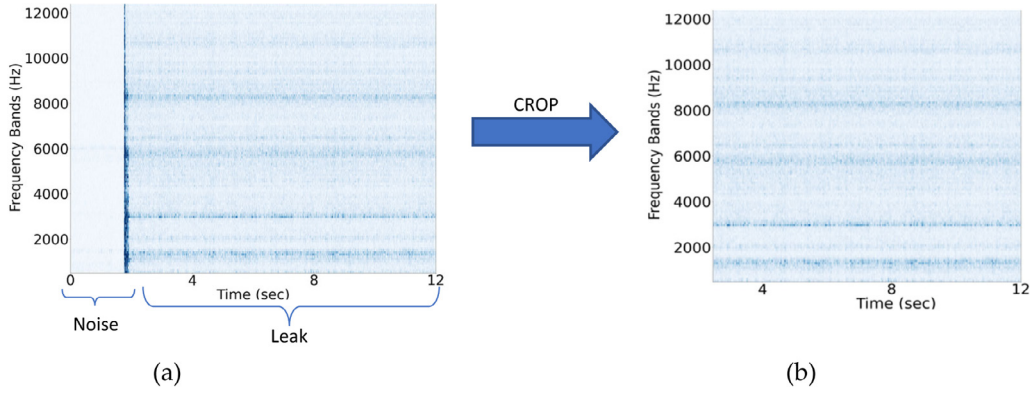


Fig. 5. (a) Illustration of a spectrogram reflecting healthy pipeline operation until approximately 2 s of monitoring and then rupture of the pipe occurred. (b) Crop of the initial spectrogram to include only the defective operation.

loss function constitutes the primary objective during the training of the networks. The Categorical Cross-Entropy (CCE) loss function was employed in our classification task. Assuming a dataset consisting of N observations, the vector containing the ground truth labels of the samples is denoted as $\mathbf{y} = [y_1, y_2, \dots, y_n]$, each assigned to 1 of a total of M labels. Additionally, following the notation previously used, the predicted values are the output values of the last layer, thus being denoted as $\mathbf{a}^{[L]} = [a_1^{[L]}, a_2^{[L]}, \dots, a_n^{[L]}]$. Therefore, the expression for the CCE loss function can be written as demonstrated in Eq. (2):

$$E = CCE(\mathbf{y}, \mathbf{a}^{[L]}) = - \sum_{i=1}^M y_i \log(a_i^{[L]}) \quad (5)$$

Thus substituting (4) into (5), the following expression is obtained:

$$E = - \sum_{i=1}^M y_i \log \left(\frac{e^{z_i^{[L]}}}{\sum_{i=1}^M e^{z_i^{[L]}}} \right) \quad (6)$$

Backpropagation: After the completion of the forward pass, the backpropagation succeeds, in the context of which the learnable parameters of the network are updated, attempting to accomplish convergence of the output predictions and the actual values of the samples, hence minimizing the loss function. For this matter, the calculation of the gradient of the loss function of each learnable parameter takes place, and it is subsequently used to update the respective parameter by an arbitrary step, determined by the learning rate, which is a hyperparameter. This process can be formulated as follows:

$$p = p - \alpha \frac{\partial L}{\partial p}, \quad (7)$$

where p represents any learnable parameter of the model and α represents the selected learning rate.

3.2. LSTM autoencoder

Long Short-Term Memory autoencoders (LSTM AE) models are trained to reconstruct the training dataset consisting of healthy signals with great precision. Conversely, in the testing phase, the model's capability to reproduce the input deteriorates significantly when facing anomalies. In essence, since the autoencoders are trained to regenerate healthy signals with great precision, the reconstruction error is anticipated to a prespecified threshold during the testing phase, signifying leakage.

3.2.1. Autoencoder

In its simplest form, an autoencoder is a class of neural networks used for the efficient reconstruction of unlabeled data. The autoencoder learns a representation for a given dataset by training the network to ignore insignificant parts of the data, such as noise. In anomaly

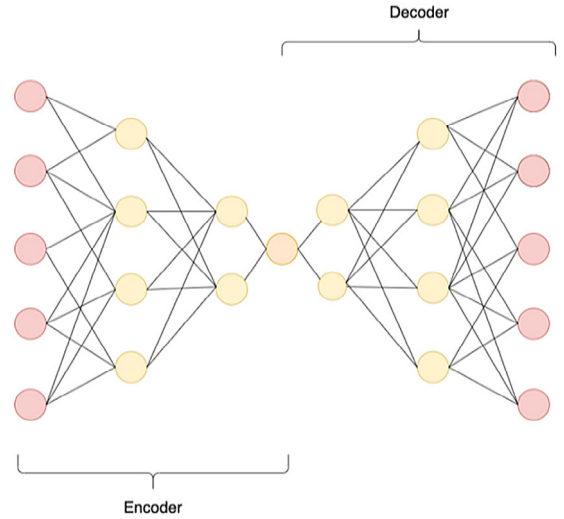


Fig. 6. Demonstration of the architecture of a typical AutoEncoder.

detection, we learn the pattern of a normal process. Anything that does not follow this pattern is classified as an anomaly. An autoencoder has two main parts, as illustrated in Fig. 6.

The first part is the encoder that maps the input into the latent representation h , and a decoder that maps the information of the latent space to a reconstruction of the input.

In the simplest case, given one hidden layer, the encoder stage of an autoencoder takes the input $\mathbf{x} \in \mathbb{R}^d$ and maps it to $\mathbf{h} \in \mathbb{R}^p$. Utilizing this information, we can express latent space h as follows:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad (8)$$

This image $\mathbf{h}$ is often referred to as latent representation or latent space. σ is an activation function such as the sigmoid or the ReLU activation function. $\mathbf{W}$ is the weight matrix, and $\mathbf{b}$ is the bias vector, usually initialized randomly, but as the training progresses, they are updated incrementally through backpropagation. Subsequently, the decoder receives the latent representation $\mathbf{h}$ and ultimately tries to reconstruct the encoder's input. In other words, the decoder attempts to map the latent representation $\mathbf{h}$ to the reconstruction $\mathbf{x}'$. Following our previous notation, this operation is formulated as follows:

$$\mathbf{x}' = \sigma'(\mathbf{W}'\mathbf{h} + \mathbf{b}') \quad (9)$$

where σ is again an activation function, and $\mathbf{W}$ and $\mathbf{b}$ are the weight matrix and the bias vector, respectively. We underline that $\mathbf{x}'$, σ' , $\mathbf{b}'$ are disparate from their encoder counterparts.

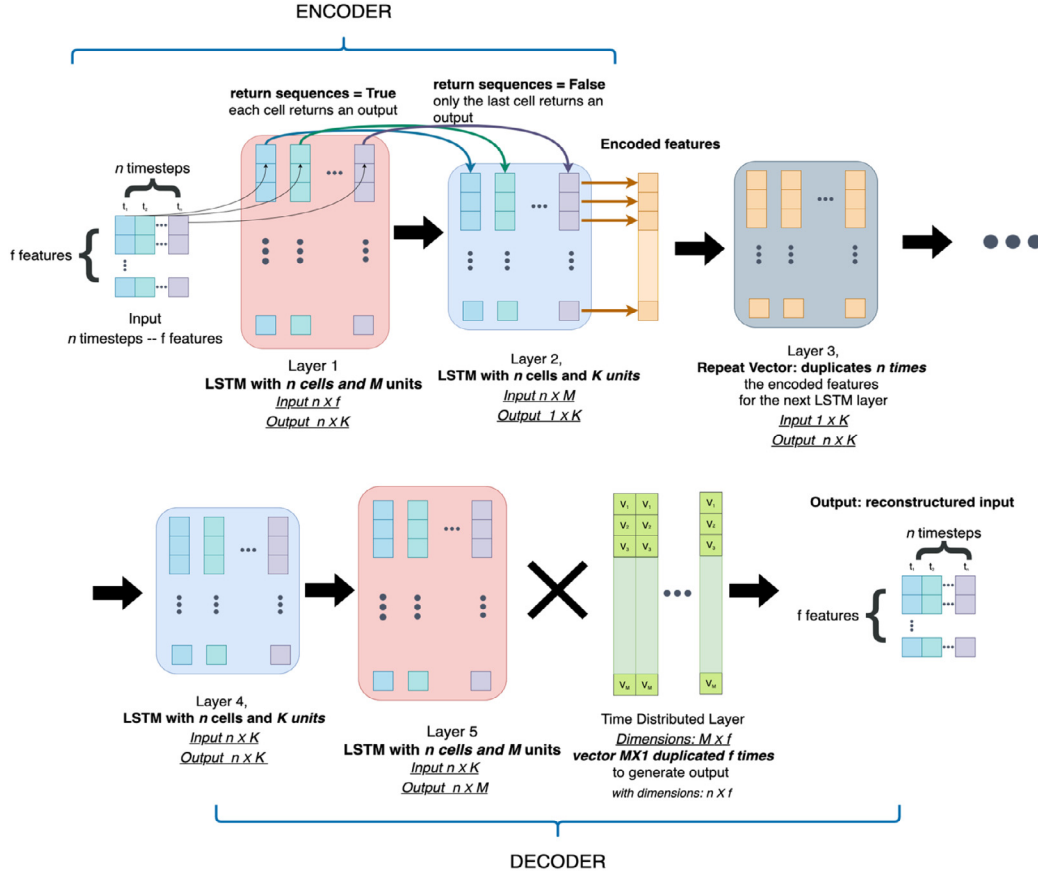


Fig. 7. LSTM Autoencoder flow diagram.

Ultimately, autoencoders are trained to minimize the loss function also referred to as reconstruction errors. For instance, an example of a reconstruction loss may be the Mean Square Error:

$$L(x, x') = \frac{1}{n} \sum_{i=1}^n (x - x')^2 = \frac{1}{n} \sum_{i=1}^n (x - \sigma'(W'(\sigma(Wx + b))) + b')^2 \quad (10)$$

3.2.2. LSTM AE

In our study case, due to the sequential trait of the signals, we develop LSTM AE which are suitable to process time series, thanks to their feedback connections, as illustrated in Fig. 7.

An LSTM AE is developed on this univariate time series to perform rare-event classification. Given a lookback window dictating the extent of the time series patch received by the network, the information flow is visualized in Fig. 7. It is observed that the LSTM network receives a 2D array with dimensions $n \times f$ as input at each timestep. The dimensions of this array correspond to the n prior timesteps the network considers at each input and the f features comprising the dataset. The LSTM layers consist of as many cells as the number of time points the network looks back at each time t . In a sequence of LSTM layers, every cell of the preceding layer generates an output to construct the 2D array the following layer requires.

To generate the reconstructed input, the output of the last LSTM layer must be multiplied by a 2D array. Essentially, this array is a vector of length equal to the number of units in each cell of the last LSTM layer, repeated f times, namely as many times as the number of features in the input. Ultimately, the goal of these networks is to minimize the divergence between the input and the reconstruction at the model's output by minimizing the reconstruction error as defined in (10). The satisfaction of this stipulation ensures the fidelity of reconstruction to the ground truth data.

The advantage of this method is that there is no need to label samples before training. More specifically, LSTM AE provides an unsupervised inspection. Our developed model is trained based on the signals of pipelines without any leakages. The objective during the training phase is to minimize the reconstruction error. Therefore, it is ensured that at the end of the training, the autoencoder is capable of reconstructing healthy signals with excellent fidelity. After the model's training, a validation set again consisting of only healthy time series separate from the training set is passed through the autoencoder to audit the model's performance. The maximum reconstruction error observed in the validation set multiplied by a safety factor serves as the leakage threshold. In essence, since the autoencoders regenerate the healthy signals with great precision, their reconstruction error is less than the threshold. Contrarily, if the reconstruction error augments significantly, exceeding this threshold during the testing phase, this event signifies leakage in the pipeline.

4. Methodology validation in a controlled environment

Validation of the proposed methodology was undertaken through parametric testing taking place at the experimental pipeline network set up for the ESTHISIS project in Kalochori, Thessaloniki, Greece. The experiments lasted two (2) days and included data collection for various leakage conditions. The controlled environment enabled the repetition of each measurement several times to eliminate statistical error. The main limitation of the experimental setup in emulating actual operational conditions was the lack of background noise, which is considerable in real setups.

4.1. Experimental setup

The experimental setup there included a water pipeline installed in open space, terminated in an engine room via which the water



Fig. 8. Leakage-inducing valves (a) and an accelerometer (b) setup along the pipeline during the field tests.

pressure inside the pipeline was monitored and regulated. Several faucets of various diameters ranging from 1 mm to 7 mm were installed alongside the pipeline, as depicted in Fig. 8(a). The majority of the runs undertaken entailed the inducement of artificial damage at the pipeline wall, emulating the leakage effect by regulating the amount of liquid exuded by the pipeline through a valve. Two sensors and the corresponding nodes were attached to the pipeline by either a screw, as shown in Fig. 8(b), or a magnetic base.

4.2. Parametric tests description

Each day of the field tests began with the sensors and nodes' installation on the pipeline. For the initial setup and after any change that altered any major geometrical parameter of the setup, such as the distance between the two sensors, the following process was followed: The water pressure was set to a pre-defined value and remained unaltered throughout the experiments, as it was regularly monitored, and water was added when needed to maintain a steady water pressure inside the pipeline. An initial series of consecutive recordings without any leaks was taken, to record a reference vibration signal for the pipeline. These recordings had a 10-min duration in total. Subsequently, a series of short tests of approximately 10 s each were carried out. During each one of these tests, one of the faucets was turned on, to emulate a leakage of a certain diameter. Each of the nodes sampled the sensors' analog signals with a sampling frequency of 25 KHz. Each run produced a time series or a sample that shall be received by the network and corresponds to 250,000-time steps, given the 10-s duration of each test and the 25 kHz sampling rate.

Preliminary signal processing is a step of great significance before feeding the signals to the networks. Real-world datasets are often incomplete, inconsistent, and likely to contain many errors. Training neural networks in raw data has been proved to be a questionable and problematic approach disorienting and misleading the optimizer, during training stage. Therefore, raw signal processing is a profoundly important matter. The methods utilized are: (i) outlier/spike removal, and (ii) data filling, and are explained in greater detail in (Theodoropoulos et al., 2021).

During the first day of the field tests in Kalochoi, the majority of the tests were carried out without water flowing inside the pipeline, whereas most of the experiments carried out during the second day involved water flowing inside the pipeline. Other parameters that changed during the field tests were: the distance between the sensors placed on the pipeline, the distance between the leakage and the sensors, and the diameter of the leakage, i.e. the diameter of the faucet that was turned on each time. Each testing procedure was repeated several times, according to the standard test practice. For the Deep Learning-based leakage detection algorithm, 58 time series were accumulated reflecting healthy pipes without any leakage and 60 time series where

Table 1

Dataset properties acquired from the experimental field tests.

Dataset properties	Value
No leak signals	58
Leak signals	60
Monitoring time per signal	10 s
Sampling frequency (Hz)	25 kHz
Signal length	250,000-time steps

Table 2

Distribution of the available signals to the training, validation, and test subsets.

Subset	Percentage	Number of healthy samples	Number of samples with leakage
Training	70%	38	37
Validation	5%	5	7
Test	25%	15	16

artificial outflow was induced were recorded during the two-day field tests. Table 1 presents the number of available signals for each major pipeline condition stemming from each experiment setting and lists general properties characterizing the time series from the methodology validation trials.

4.3. Results

4.3.1. 2D-CNN classifier

For the utilization of this classifier, it is intrinsic to convert the time series into spectrograms so that the network receives the images it requires. Examples of spectrograms are illustrated in Fig. 9. As observed, the measurements collected by the sensor are mainly noise in the case where the pipeline indicates no signs of leakage. Conversely, in the case of leakage, some frequency bands seem to predominate, as they appear in darker blue. The challenge lies in the fact that leakage with a smaller diameter generates a spectrogram similar to a spectrogram corresponding to a well-functioning pipeline. Thus, potentially confounding our CNN classifier. Each time series, described in Table 1, generates 1 spectrogram, and the distribution to the training, validation, and test set are tabulated in Table 2.

In this stage of our analysis, we are more interested in identifying leakage rather than recognizing the leakage's diameter. Thus, we opt to contract our problem into binary leakage classification since the division of the samples in the corresponding diameters is very refined namely, there is a paucity of signals for each distinct diameter. However, regardless of our approach, it is essential to ensure that each state and diameter is equally represented in each of these subsets (training-validation-test) to avert skewed training. Table 3 demonstrates how the samples corresponding to each diameter were distributed to each subgroup.

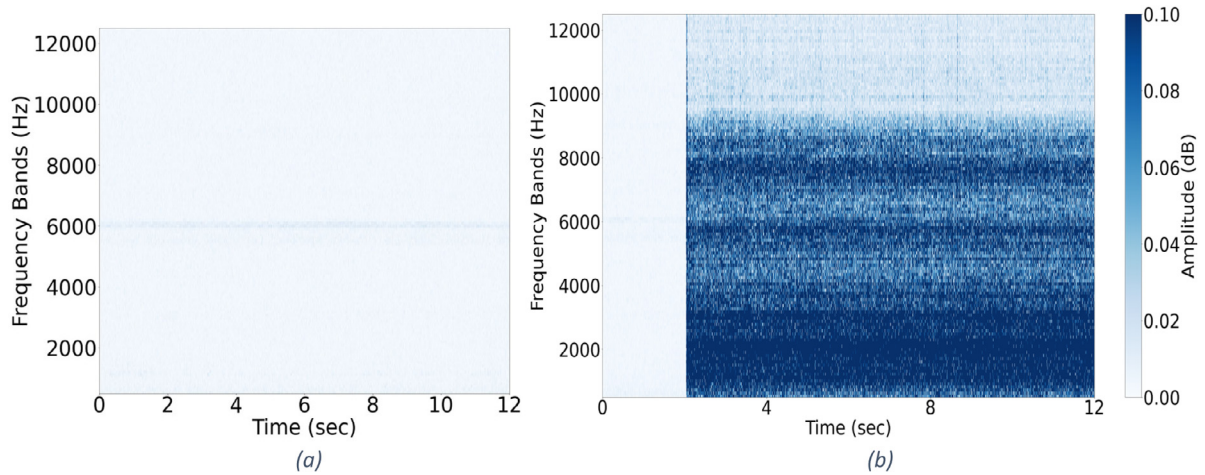


Fig. 9. Comparison of the spectrograms generated by signals reflecting different conditions. (a): Healthy, (b): 7 mm Leakage diameter.

Table 3

Number of available defective signals and their distribution.

Diameter	Distribution (Total: Training-Validation-Test)
1 mm	7: 4-1-2
2 mm	7: 4-1-2
3 mm	10: 7-1-2
4 mm	8: 5-1-2
5 mm	10: 6-1-3
6 mm	10: 6-1-3
7 mm	8: 5-1-2

Table 4

CNN-classifier architecture and hyperparameter selection.

Name of component	Value
Convolutional layer #1	$256 \times 256 \times 15$, Kernels: 3×3
Convolutional layer #2	$128 \times 128 \times 6$, Kernels: 3×3
Max pooling layer #2	$64 \times 64 \times 6$, Kernels: 2×2
Convolutional layer #3	$64 \times 64 \times 6$, Kernels: 3×3
Max pooling layer #3	$32 \times 32 \times 6$, Kernels: 2×2
FCN layer #1	3 nodes, Dropout = 0
Output layer	2 nodes
Learning rate	$5 \cdot 10^{-4.0}$
Weight updates	Epochs•BatchSize = $8 \cdot 100 = 800$

As already mentioned, in the training and test samples fed to the classifier, we purposefully omit the time point where efflux occurs to examine the efficacy of the proposed methodology to identify leakage on the occasion that at the time of the trigger event, no monitoring was carried out. Indicative, we present a sample cropped to incorporate only the defective condition. The training set consists of the images dedicated to the model's learning process. Provided that this is the most challenging and demanding procedure, inherently, it requires the most samples. The validation set is utilized to track the training progress of the classifier while also is utilized to assess the different configurations of hyperparameters during the tuning process. Lastly, the test set comprises previously unseen image instances to examine the ability of our classifier to generalize. It is recommended to explore the capacity for generalization of the models using an adequate number of samples. Upon this reasoning, the percentage for distributing the entirety of the images across the 3 subsets is 70%-5%-25%, respectively. For the network's architecture, the 2D-CNN classifiers presented in (Lu et al., 2017) served as our starting point. After experimentation with the values of the hyperparameters, the configuration listed in Table 4 was concluded.

Consequently, the efficacy of the proposed methodology is evinced in Fig. 10 and Tables 5 and 6. More specifically, the optimized proposed classifier correctly categorized every instance in the test set. Explicitly,

the left part of Fig. 10 illustrates the ROC curve and the respective Area Under Curve (AUC) of the optimal CNN classifiers that occurred after the hyperparameter tuning stage. Additionally, the right part of Fig. 10 demonstrates the accuracy and loss function of the optimal model during its training. Furthermore, in Tables 5 and 6 we juxtapose the classifier instances during the hyperparameter tuning phase. More specifically, if the 353 cases defined by the different hyperparameter configurations were ranked according to their accuracy in correctly classifying the test set samples, we tabulate the confusion matrices of the instances appertaining to the 20th, 50th, 80th, and 100th percentile. Additionally, in the aforementioned Tables, we list several performance metrics, such as the accuracy, the loss function value, and the respective AUC of the aforementioned instances. It can be observed that the optimized model achieved perfect accuracy of the samples in the test set and that more than 50% of the instances achieved an AUC greater than 0.8 and respectively yielded accuracy greater than 80%, indicating relative immunity of the model to various hyperparameter arrangements.

4.3.2. LSTM AE

Furthermore, we also proceed to demonstrate the results yielded by the LSTM autoencoders. After applying normalization of the signals, we present examples of signals corresponding to the healthy state and leakage of 7 mm in Fig. 11. The Simple Moving Average with a rolling window of 10-time steps was applied to remove some noise, thus observing the plots better.

As observed, in the case that the sensor monitors an impermeable pipeline, the collected signal resembles a sinus plot. Additionally, it may also be observed that the greater the leakage diameter the greater the impact on the signal. Lastly, it is evinced that in the case of small leakage diameters (i.e., 1–2 mm), the impact of the efflux occurrence is small. More specifically, this trigger event invokes a small region where the signal is considerably augmented; nevertheless, it quickly returns to its initial condition. This fact omens that the LSTM autoencoder will be capable of identifying the occurrence of the trigger event but will likely struggle to continue notifying the pipeline functioning as defective.

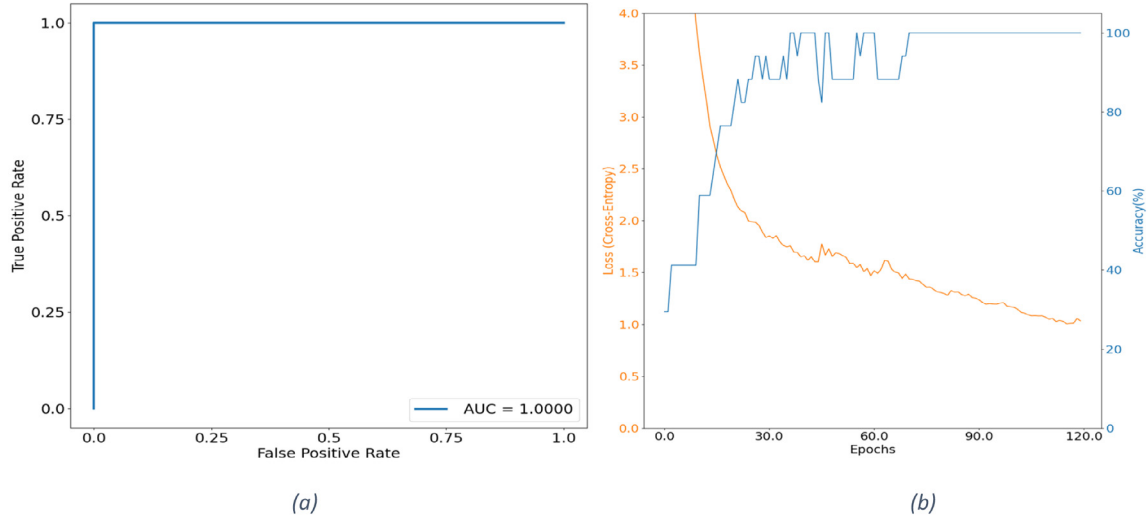
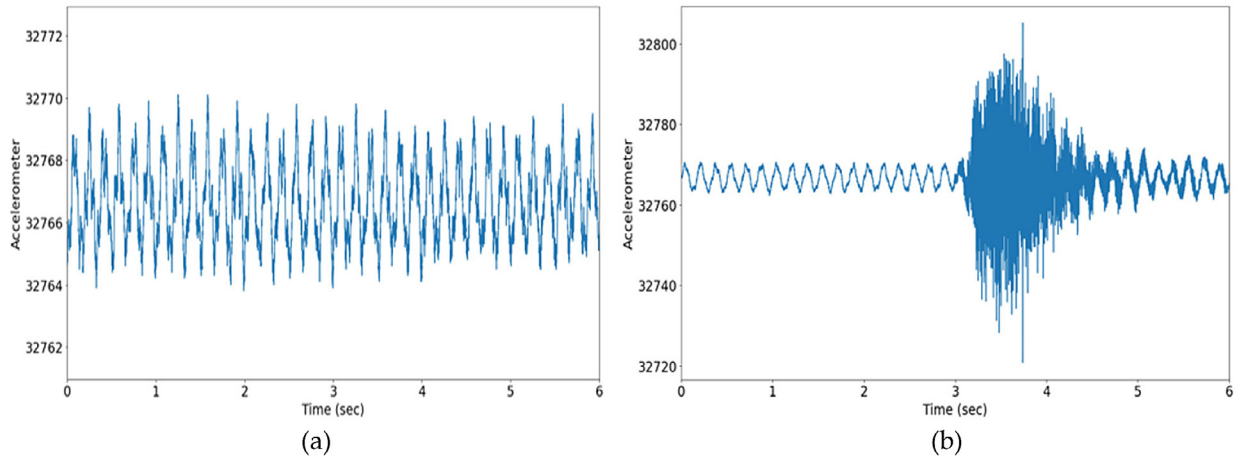
Subsequently, Table 7 lists the distribution of the samples in the respective subsets for the implementation of the LSTM autoencoders. As described above, the training set consists of the samples necessary for the model to learn how to reconstruct the input signal with high fidelity. Since it is desired for our model to learn how to reconstruct healthy signals, it receives in the training phase only healthy samples. The validation set comprises only healthy signals since it generates the leakage detection threshold. Lastly, the test set primarily consists of defective signals, namely signals incorporating the emanation time point, to validate timely detection of leaks.

Table 5

Confusion matrices of the 4 instances of the CNN classifier.

		20th percentile		Median		80th percentile		Optimal (100th pct)	
		L	NL	L	NL	L	NL	L	NL
Predicted	L	10	5	13	0	15	0	16	0
	NL	8	8	5	13	2	14	0	15

*L: LEAK, NL: NO LEAK.

**Fig. 10.** (a): ROC curve of the CNN classifier, (b): Loss and Accuracy of the CNN classifier during training.**Fig. 11.** Examples of signals representing different conditions. (a): Healthy, (b): 7 mm Leakage diameter.**Table 6**

Performance of the 4 instances.

	20th percentile	Median	80th percentile	Optimal (100th pct)
Loss	1.024	0.951	0.873	0.392
Accuracy (%)	58.3	83.8	93.5	100
AUC	0.65	0.83	0.90	1.00

Table 7

Distribution of the available signals to the training, validation, and test subsets.

Condition	Training	Validation	Test
Healthy samples	70% - 41 samples	15% - 8 samples	15% - 8 samples
Samples with leakage	0%	0%	100% - 60 samples

For the architecture of our LSTM autoencoders, extensive experimentation of the hyperparameter configuration was carried out, and it was concluded that optimal performance was achieved while applying a 3 time-steps “lookback” window, and the implemented architecture entailed 2 encoding hidden layers with 128 and 64 LSTM units and respectively 2 decoding with 64 and 128 units. Table 8 summarizes the architecture of the LSTM autoencoders and the selection of the hyperparameters.

For the continuous notification of the malfunction, the following assumption is made: if out of the 25,000 measurements collected in 1 s, the number of observations above the leakage threshold is greater than 10,000, then the system is flagged as defective. The number “10,000” is arbitrarily selected, and it manifests our emphasis towards averting false negatives, namely indications where the system does not identify potential system defectiveness. However, this process also protects us from false positives in the event of erroneous measurement, which

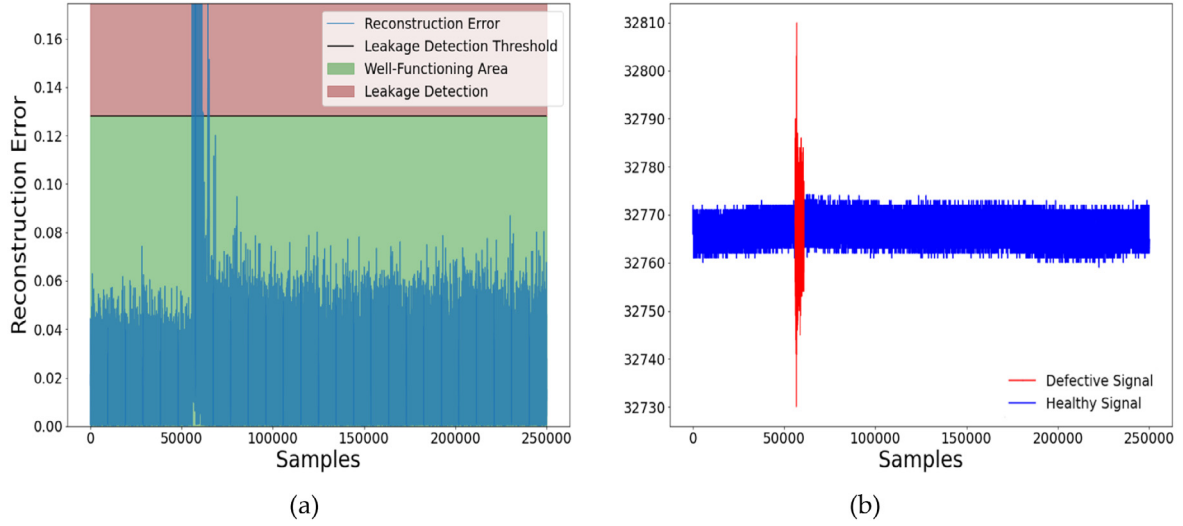


Fig. 12. Defective signal—1 mm leakage diameter. (a): Reconstruction error of the LSTM autoencoder. (b): Acoustic signal, with blue the healthy signal is represented, and red, the signal after the leakage.

Table 8

LSTM autoencoder architecture and hyperparameter selection.

Name of component	Value
LSTM layer 1 units (Encoding 1st–Decoding 2nd)	128
LSTM layer 2 units (Encoding 2nd–Decoding 1st)	64
Learning rate	2×10^{-4}
“Lookback” window	5
Epochs	200
Batch size	8

would lead the autoencoder to exceed the reconstruction threshold. Denoting with 1 and 0 the Boolean variables True and False, respectively, we formulate our decision process in Eq. (11).

$$\begin{aligned}
 \text{Possible Leakage} &= \begin{cases} 1 & \text{if } \text{Error} > \text{Threshold} \\ 0 & \text{if } \text{Error} < \text{Threshold}, \end{cases} \\
 \text{Leakage Prediction} &= \begin{cases} 1, & \text{if } \sum_{i=t}^{t+25,000} \text{Possible Leakage}_i > 10,000 \\ 0, & \text{if } \sum_{i=t}^{t+25,000} \text{Possible Leakage}_i < 10,000 \end{cases}
 \end{aligned} \quad (11)$$

Lastly, the performance of these models to detect outflow is presented. Indicatively, Figs. 12 and 13 demonstrate the detection of the pipeline malfunction through the threshold generated by the network on two separate occasions, explicitly while attempting to identify 1 mm and 4 mm leak diameter, respectively.

The left portion of the Figure illustrates the reconstruction error and the portion of the signal generating reconstruction greater than the leakage threshold. The right portion represents the actual acoustic signal as acquired from the monitored pipeline. The blue part of the signal corresponds to the healthy condition of the channel, whereas with red, we denote the outflow.

It is manifested that in small leakages where the impact of the efflux is relatively confined to a small region in the signal, thus the efficacy of the LSTM autoencoder to flag the leakage continuously is limited. It can be observed that although these networks seem to identify correctly and timely even minimal leakage diameters, they do not keep identifying the system as defective. One possible explanation is that after the trigger event effectuating the efflux, the signal appears almost identical to its form before that event, namely the healthy state,

Table 9

LSTM AE detection confusion matrix.

		True Class	
		No Leak	Leak
Predicted	No Leak	63	9
	Leak	2	55

causing the model to struggle. Thus, we can discern a weakness of these models, as it is possible for leakages to remain undetected in the event of not monitoring during the incidence of the leakage is eminent. The confusion matrix of Table 9 summarizes the efficiency of the LSTM autoencoder in detecting the leakage event and subsequently continues to notify potential stakeholders regarding the impairment of the monitored system. Since the signals representing leakage in our experiments in Kalochori incorporate both conditions, namely merely noise at the beginning of each trial and subsequently outflow, each such signal is represented in the confusion of matrix of Table 9 twice to examine if each state is represented correctly.

It is demonstrated through Table 9, as well through Figs. 12 and 13, that this methodology, despite its weakness, to continuously flag the condition as defective for small leakage diameters, in general, detects with remarkable accuracy, and it also offers the possibility for immediate notification of the emergence of the leakage.

5. Evaluation in a real environment

Evaluation of method efficiency in a real case scenario was conducted in the facilities of an oil refinery (Hellenic Petroleum) in Greece. The tests lasted three days and while, due to safety reasoning, parametric testing was limited, the tests were performed during normal operation of the pipeline.

5.1. Experimental setup and measurement process

The experimental setup for the field tests in a real environment was similar to the one described Section 4.1 for the field tests in Kalochori. The factors discerning the two setups were the pipeline diameter, with the pipeline in this setup being substantially larger,

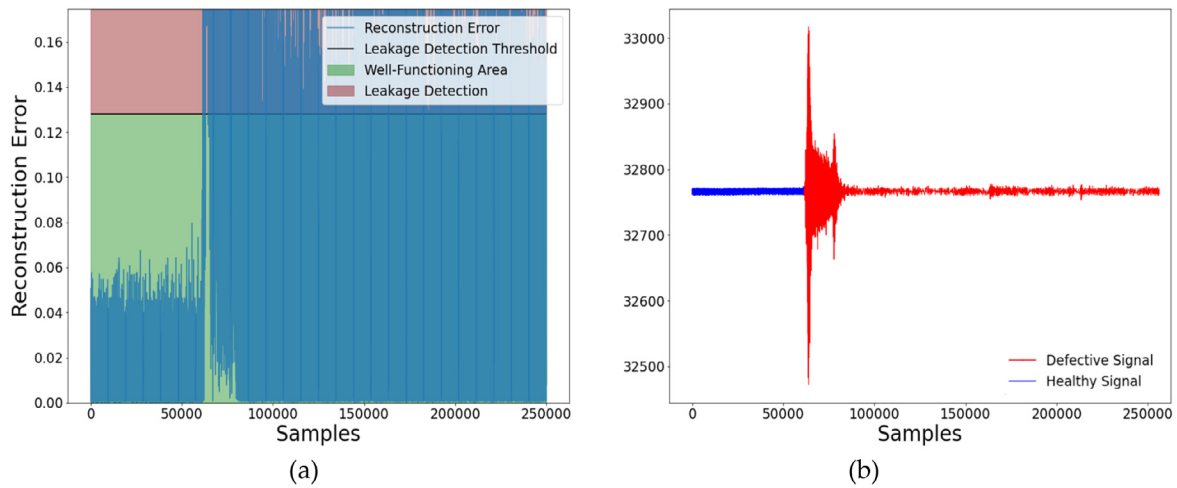


Fig. 13. Defective signal—4 mm leakage diameter. (a): Reconstruction error of the LSTM autoencoder. (b): Acoustic signal, with blue the healthy signal is represented and with red the signal after the leakage.

Table 10

Dataset properties acquired from the pilot field tests.

Dataset properties	Value
Healthy signals	27
Defective signals	23
Monitoring per signal	10 s
Sampling frequency (Hz)	25 kHz
Signal length	250,000-time steps

the water pressure inside the pipeline, and the available valves for artificial leakage creation, with the available diameters being 5 mm and 13 mm. The process followed the initial setup or any change in the setup geometry was also the same as the one described in Section 4.2, except for the fact that liquid pressure inside the pipeline was not controlled; the remaining parameters and processes remained the same.

For this DL-based leakage detection, the field-testing team collected 27 signals reflecting “healthy” pipes without any leakage, and 23 samples where artificial outflow was induced; 19 of them imitated the effect of a 13 mm leakage diameter and the remaining 4 emulated a smaller 5 mm diameter. The challenge of the analysis of the signals collected in a real operating environment lies in the considerable ambient noise extant in the measurements that stem from other ongoing procedures taking place at the facilities, instigating external noise entering the monitored system. The signals underwent preliminary signal processing for the same reasons as explicated in Section 4.2. The steps followed are identical, namely: (i) outlier/ signal spike removal, and (ii) blank spots filling. Table 10 summarizes the properties of the signals acquired during the pilot trials, such as the number of available signals, and general properties characterizing each time series.

5.2. Results

5.2.1. CNN-2D classifier

After the signal acquisition, converting the time series to spectrograms is repeated. Examples of spectrograms from this dataset are demonstrated in Fig. 14.

In comparison with the field testing and the respective spectrograms, it is easily observed through the amplitude scale that significantly more noise enters the measurements, emulating actual operating conditions. Thus, the challenge of this analysis lies in achieving train models capable of distinguishing leakage despite the extraneous noise from the pipeline’s surroundings. The distribution to the training, validation and test set are tabulated in Table 11 below. The assignment of the leakage reflecting signals to each subset is shown in Table 12.

Table 11

Distribution of the available signals from the pilot testing to the training, validation and test subsets.

Subset	Percentage	Number of healthy samples	Number of samples with leakage
Training	70%	15	15
Validation	5%	3	3
Test	25%	9	5

Table 12

Number of available defective signals and their distribution.

Diameter	Distribution (Total: Training-Validation-Test)
5 mm	4: 2-0-2
13 mm	19: 13-3-3

Table 13

CNN-classifier architecture and hyperparameter selection.

Name of component	Value
Convolutional layer #1	256 × 256, Kernels: 3 × 3
Convolutional layer #2	32 × 32, Kernels: 3 × 3
Max pooling layer #1	32 × 32, Kernels: 2 × 2
Convolutional layer #3	32 × 32, Kernels: 3 × 3
Max pooling layer #2	32 × 32, Kernels: 2 × 2
FCN layer #1	15 nodes, Dropout = 0.3
Output layer	2 nodes
Learning rate	5·10 ^{-4.0}
Weight updates	Epochs·BatchSize = 8·100 = 800

While emulating defective pipelines, the measurements always started after having already turned on the valves allowing the outflow thus there was no need to crop the dataset as in our previous study case. For the network’s architecture, starting again from the 2D-CNN classifiers of (Theodoropoulos et al., 2021), the configuration listed in Table 13 was concluded after implementing the Grid Search algorithm for extensive inquiry of the optimal configuration.

Consequently, the performance of the proposed methodology in the dataset obtained from our pilot testing is demonstrated in Tables 14 and 15, as well as in Fig. 15. Similarly, on this occasion, the optimized proposed classifier correctly categorized every instance in the test set. The left part, Fig. 15 explicitly illustrates the ROC curve and the respective Area Under Curve (AUC) of the optimal CNN classifiers that occurred after the hyperparameter tuning stage. Additionally, the right part of Fig. 15 demonstrates the accuracy and loss function of the optimal model during its training. Following the same pattern as in our previous study case, in Table 15, we compare the performance of

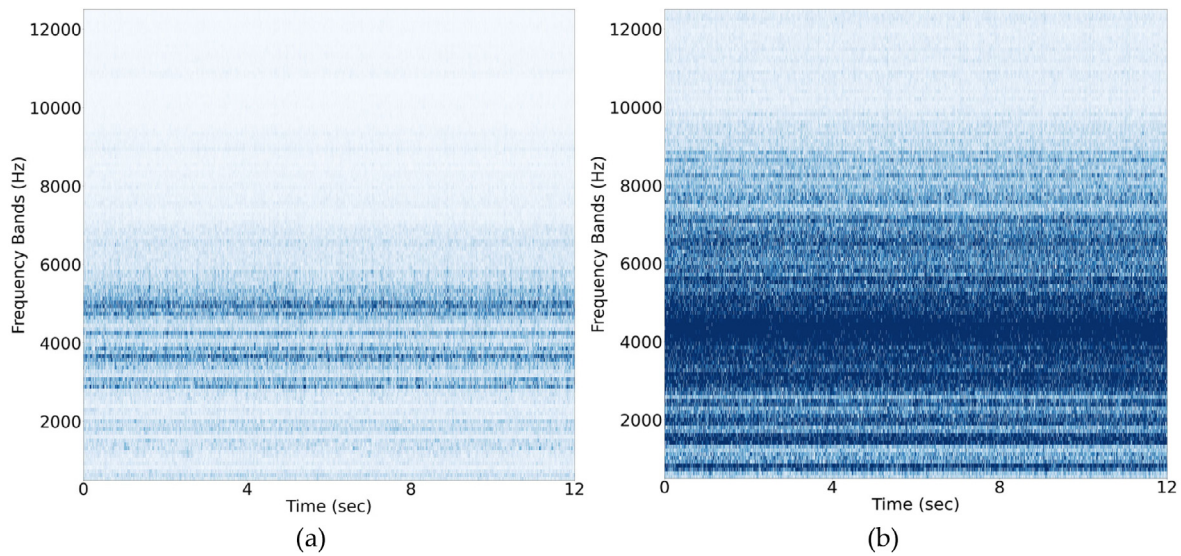


Fig. 14. Comparison of the spectrograms generated by signals from the pilot testing reflecting different conditions. (a): Healthy, (b): 13 mm Leakage diameter.

Table 14
Confusion matrices of the 4 instances of the CNN classifier.

		20th percentile		Median		80th percentile		Optimal (100th pct)	
		L	NL	L	NL	L	NL	L	NL
Predicted	L	5	2	5	3	5	1	5	0
	NL	3	4	0	6	0	8	0	9

*L: LEAK, NL: NO LEAK.

Table 15
Performance of the 4 instances in the pilot testing dataset.

	20th percentile	Median	80th percentile	Optimal (100th pct)
Loss	6.42	3.53	2.09	1.72
Accuracy (%)	64.2	78.6	92.8	100
AUC	0.64	0.74	0.87	1.00

classifier instances appertaining to the 20th, 50th, 80th, and 100th percentile from the hyperparameter tuning step, presenting their confusion matrices, and listing the same performance metrics to comprehend the efficacy of these instances.

It can be observed that despite the small number of available samples, not only does our proposed methodology achieve perfect accuracy of the samples in the test set, but the majority of the instances display satisfactory performance, suggesting immunity of the proposed methodology to different hyperparameter arrangements. Therefore, the proposed classifier offers an effective way to determine the condition of the monitored pipelines by converting the signals into spectrograms.

5.2.2. LSTM AE

Examples of signals corresponding to the healthy state, and leakage of 13 mm, are illustrated in Fig. 16. The Simple Moving Average with a rolling window of 10-time steps was applied to remove some of the noise, thus better observing the plots. Similarly, with the previous experiments, it is observed that the greater the leakage diameter, the more significant the impact on the signals. Subsequently, Table 16 lists the distribution of the samples in the respective subsets for the implementation of the LSTM autoencoders.

As described above, the training set consists of the samples necessary for the model to learn how to reconstruct the input signal with excellent fidelity. Since it is desired for our model to learn how to reconstruct healthy signals, it receives in the training phase only healthy samples. In the present study case, we opt not to use a validation set due

Table 16
Distribution of the available signals to the training, validation, and test subsets.

Condition	Training	Validation	Test
Healthy samples	70%- 20 samples	0%	30%-7 samples
Samples with leakage	0%	0%	100%-23 samples

Table 17
LSTM autoencoder architecture and hyperparameter selection.

Name of component	Value
LSTM layer 1 units (Encoding 1st–Decoding 2nd)	128
LSTM layer 2 units (Encoding 2nd–Decoding 1st)	64
Learning rate	2×10^{-4}
“Lookback” window	5
Epochs	200
Batch size	8

to the want of samples. The threshold shall be computed by repassing the training through the model to generate inferences based on this set. Lastly, the test set is primarily composed of defective signals, namely signals incorporating the emanation time point, to validate the timely detection of leakage.

For the architecture of our LSTM autoencoders, extensive experimentation of the hyperparameter configuration was carried out. It was concluded that optimal performance was achieved while applying a 5 time-steps “lookback” window. The implemented architecture entailed 2 encoding hidden layers with 80 and 60 LSTM units and respectively 2 decoding with 60 and 80 units. Table 17 summarizes the architecture of the LSTM autoencoders.

Finally, using the same decision process as in our previous study case, we observe that the LSTM autoencoder correctly identified the healthy signal of Fig. 17 since its reconstruction error was below the detection threshold. Likewise, the defective signal was also correctly diagnosed, as depicted in Fig. 18.

The confusion matrix of Table 18 summarizes the efficiency of the LSTM autoencoder. It is evinced that in this study case, despite not

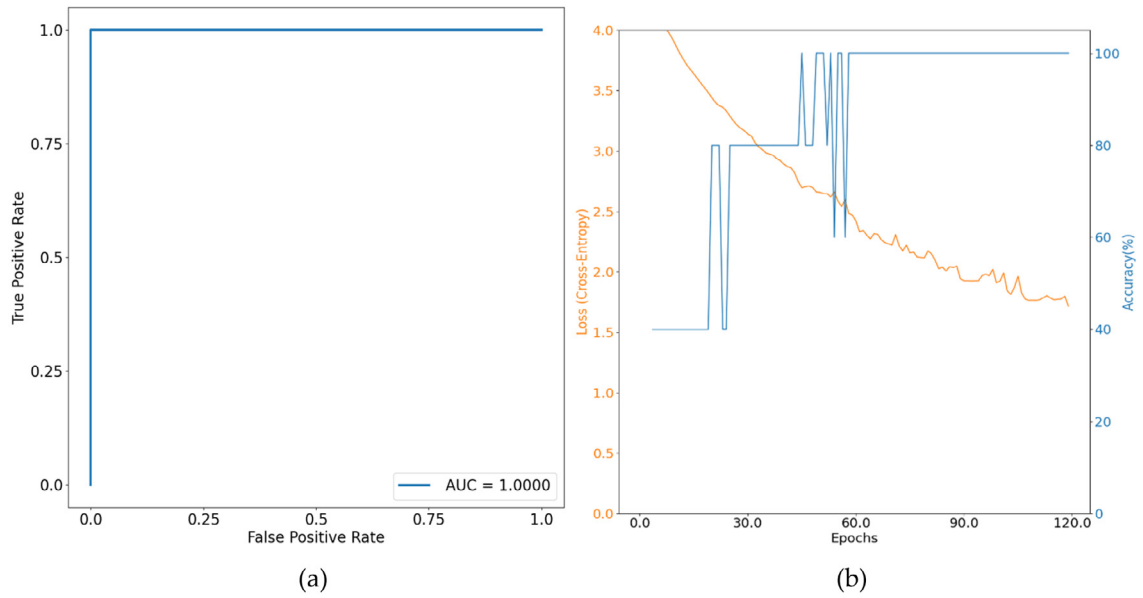


Fig. 15. a: ROC curve of the CNN classifier, b: Loss and Accuracy of the CNN classifier during training.

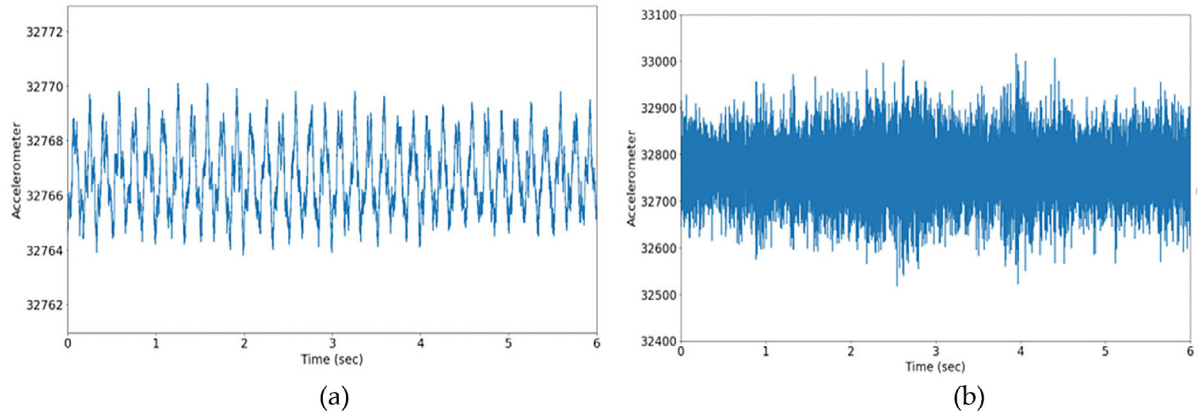


Fig. 16. Examples of signals representing different conditions. (a): Healthy, (b): 13 mm Leakage diameter.

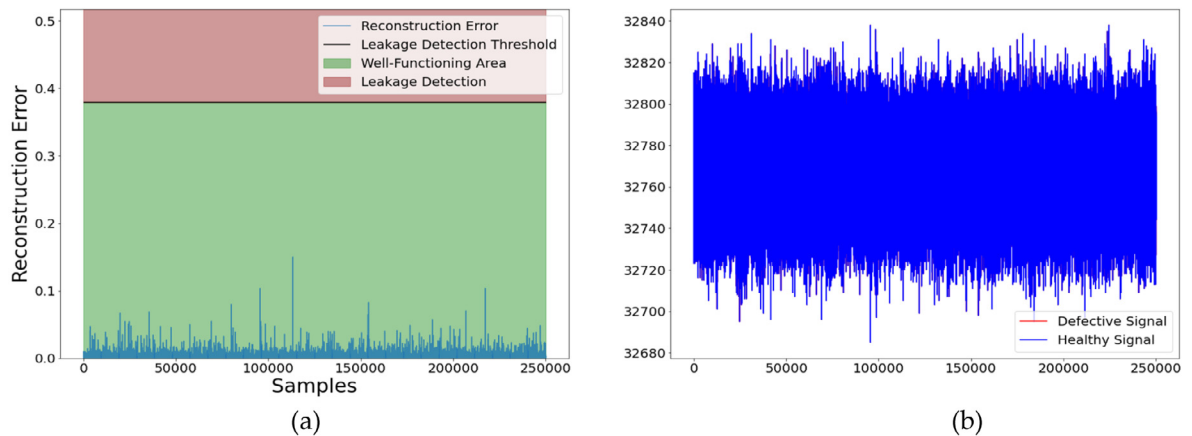


Fig. 17. Healthy signal. (a): Reconstruction Error of the LSTM autoencoder. (b): Acoustic signal, with blue the healthy signal, is represented and with red the signal after the leakage.

having a plethora of samples available to train and test the model, remarkable performance has been achieved as it correctly detects leakage at every defective signal. Likewise, it correctly did not identify faulty

operational behavior when tested in healthy signals. Therefore, the proposed LSTM autoencoder offers an effective way to identify leakages in the monitored pipelines immediately.

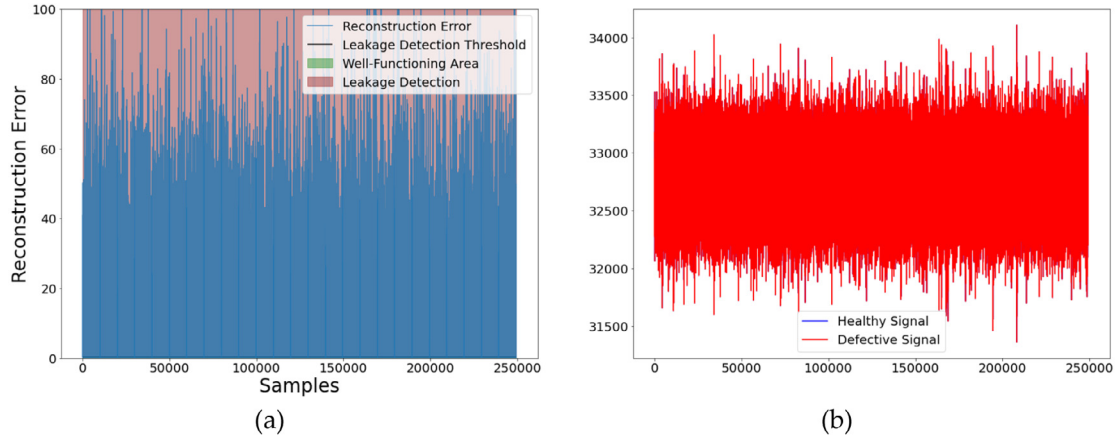


Fig. 18. Defective signal—13 mm leakage diameter. (a): Reconstruction Error of the LSTM autoencoder. (b): Acoustic signal, with blue the healthy signal, is represented and with red the signal after the leakage.

Table 18
LSTM AE detection confusion matrix.

		True Class	
		Negative Labeled	Positive Labeled
Predicted	Negative	27	0
	Positive	0	23

6. Comparison with other methods

Despite, having established the efficacy of the proposed methodology in the experimental as well as in the actual pipeline setup, it is intrinsic to compare our proposed models to other algorithms widely employed in pertinent literature for the task of anomaly detection, which thanks to the vast amount of available data has steeply progressed employing data-driven techniques. The most common categories of utilized methods for anomaly detection in univariate time-series are: (i) Statistical approaches, (ii) Ensemble training, (iii) Domain based methods (Basora et al., 2019). In this manner, a better understanding of the task at hand is provided and importantly, the efficacy and novelty of our methodology shall be better assessed. For this matter, 3 models appertaining to classical machine learning and statistical methods were implemented:

1. Support Vector Machine (SVM)

The support vector machine algorithm was first invented as a linear classification approach and was subsequently extended to perform nonlinear classification by integrating kernel operations to project the samples from the input space X into another input space V , to avoid explicit mapping required by linear learning algorithms to learn nonlinear decision boundaries. This technique was further expanded to detect “novelties” through the One-Class SVM variation. OC-SVM is a semi-supervised approach, receiving only healthy data during the training phase. After the model has been trained, the samples on the test data are evaluated regarding their proximity to the training data, enabling the model to detect abnormalities.

Initially, this class of anomaly detection algorithm was employed to detect anomalies in a set of vectors and not in time series (Braei and Wagner, 2020). For this matter the transformation of time series into a vector set has been proposed. More specifically, several studies (Zhang et al., 2008, 2007) have posed the creation of rolling windows with length w time points, so that the

initial time series of length $X = (X_1, \dots, X_t)$ is converted into: $W = (W_1, \dots, W_p) \Rightarrow W = ((X_1, \dots, X_w)^T, (X_2, \dots, X_{w+1})^T, \dots, (X_p, \dots, X_{w+p-1})^T)$. Then a function f projects the univariate time series into a two-dimensional space:

$$f: \mathbb{R} \rightarrow \mathbb{R}^2, f = \begin{cases} X_t X_t & \text{if } t = 1 \\ [X_{t-1} X_t] & \text{else,} \end{cases} \quad (12)$$

2. Isolation Forest (iForest)

Isolation Forest (iForest) is an ensemble learning algorithm. The concept of Isolation Forest is based on two characteristics and assumptions regarding abnormal data: (i) anomalous behavior can be easily located as an outlier, and (ii) abnormal data is merely a small proportion to the entire dataset. In similar fashion to random forests, that are composed of a large number of decision trees, an iForest also consists of a large number of trees called isolation trees (iTrees).

For the construction of an iTree, a subset of the data points from training data are randomly selected, constituting a sample subspace. Subsequently, a feature of the sample subspace is randomly selected and a cutting value V of that feature. The samples of the subspace are segregated into two parts with respect to the cutting value V . Assuming, a leaf node T of the isolation tree, the observations whose value at this specified feature is less than V are assigned to the left child node; respectively if the value of the observation is greater than V , the data is assigned to the right subtree. This process is repeated recursively until each point is isolated (Qin and Lou, 2019).

Secondly, the anomaly score of each sample x is calculated, given its path length $h(x)$. The path length of a sample is defined as the number of edges traversing from the root node to an external node. This method considers that points with shorter paths are easier to isolate and so they should have shorter path lengths than normal instances (Zhong et al., 2019). For a sample subspace with size n and x as one instance, the anomaly score s of an instance x is defined as:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}, \quad (13)$$

where

$$c(n) = 2 \cdot H(n-1) - \frac{2(n-1)}{n} \quad (14)$$

with $H(n)$ is the harmonic number and can be approximated by $\ln(n) + \eta$, (η : Euler's constant). The quantity $c(n)$ denotes the average $h(x)$ given n and is used to normalize $h(x)$. The likelihood of a sample being anomalous increases as its anomaly score approaches 1. If the anomaly score is close to 0.5, we do

not possess enough evidence to ascertain whether it is an outlier. Lastly, if the anomaly score is below 0.5, then the observation is deemed as normal. For the hyperparameters of the Isolation Forests, the default values suggested by the scikit-learn library were used.

Finally, this method receives a time series as was previously shown for the OC-SVM, namely considering a window of length w data points, rolling through the initial time series: $X = (X_1, \dots, X_t)$ into $W = ((X_1, \dots, X_w)^T, (X_2, \dots, X_{w+1})^T, \dots, (X_p, \dots, X_{w+p-1})^T)$.

3. ARMA and Rolling z-score

The AutoRegressive Moving Average model of univariate stationary time series combines:

- The AR model where the current value X_t of the stochastic process (dependent variable) is based on a finite set of previous values (independent variables) of length p :

$$X_t = \sum_{i=0}^p a_i X_{t-i} \quad (15)$$

- The MA which considers the current observation X_t as a linear combination of the last q prediction errors:

$$X_t = \sum_{i=0}^q b_i \epsilon_{t-i} \quad (16)$$

- Thus, the ARMA model can be expressed as follows:

$$X_t = \sum_{i=0}^p a_i X_{t-i} + \sum_{i=0}^q b_i \epsilon_{t-i} \quad (17)$$

The values of the coefficients $a_1, \dots, a_p$ and $b_1, \dots, b_q$ are evaluated during training.

These methods are based on a regression model is first fitted on the training data. Then the resulting model is used to forecast test sequences and the difference between the predicted and real values is called residual. Suppose the orders p and q of the AR and MA models respectively have been chosen appropriately to model the given time series. In that case, it follows that the residuals are assumed to be distributed normally. Subsequently, these residuals are utilized to calculate the rolling z-score of the prediction error. Assuming that the fitted model is capable to predict satisfactorily the healthy time-series provided in the training step, if the error continuously exceeded the 95% confidence interval it would serve as an anomaly indicator since the model would fail to predict the time series accurately based on the system dynamics learned during training signifying a significant change in the system. The input time series were ascertained to be stationary through the Augmented Dickey Fuller test for our problem. The AR and MA order found from the training of the ARMA models to adequately capture the piping system's dynamics were $p = 4$ and $q = 5$.

The aforementioned methodologies are semi-supervised or unsupervised, as the training data consists of only healthy data, exempting us from having to label the data of the dataset. This convenience prompts researchers to adopt this approach frequently for the task of anomaly detection in univariate time-series, as the model is trained to fit the distribution of the healthy time-series and subsequently any point deviating from this distribution in the test set is identified as an anomaly. Given the possession of common features and attributes of all 3 of the methods presented above with the proposed LSTM AE, the same detection scheme is implemented, as described in Section 4.3.2, in order to identify a sample as defective. More specifically, if out of the 25,000 measurements collected in 1 s, the number of observations above the leakage threshold is greater than 10,000, then the system is flagged as defective. Each sample represents 10 s of operational data.

The training and testing subsets for each benchmark model are identical to the counterpart subsets used for the LSTM AE.

Furthermore, diverse metrics provide a more comprehensive understanding of the different models' performance. Provided that our dataset does not consist of equally healthy and degraded signals enable us to identify better the deficiencies of each model. In essence, our problem may be approached as a binary classification. Each sample is evaluated whether leakage has occurred during the timeframe reflected in the respective signal. The metrics employed are as follows:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}, \quad (18)$$

$$Precision = \frac{TP}{TP + FP}, \quad (19)$$

$$Recall = \frac{TP}{TP + FN}, \quad (20)$$

$$Specificity = \frac{TN}{TN + FP}, \quad (21)$$

$$F1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}, \quad (22)$$

where TP, TN, FP, FN denotes the True Positive, True Negative, False Positive and False Negative respectively.

It is reminded that from the hyperparameter tuning of our proposed models, the instances occurring from this procedure were ranked with regards their accuracy. In this point, instead of comparing the optimal instances of the proposed networks to the benchmark algorithms, the instances appertaining to the 80th percentile are employed in order to offset the lack of extensive optimization and tuning for the ML models and the ARMA approach.

Observing Table 19 provides the conclusive results regarding the comparison of the posed neural networks, with other well-established methods in the field of anomaly detection. More specifically, it is presented that from the benchmark ML methods the OC-SVM and the Isolation Forests were susceptible to falsely detect leakages in healthy samples. This is demonstrated by the relatively lower precision and specificity values. Additionally, the ARMA method provided better results than the other benchmark methods, showcasing the efficacy of these models to process time series signals, however they still underperformed compared to the 2 proposed deep learning models. Moreover, it is also worth noting that the training time for the ARMA model including the tuning for the appropriate p and q values was considerably longer than the tuning phase for each suggested neural network. Running on Apple Silicon M1 processor, using the integrated graphics card the CNN Classifier was optimized in 1 h and 13 min inspecting over 20 models and the LSTM AE was optimized in 56 min inspecting the performance of 15 models. Conversely, the ARMA models run on the M1 integrated CPU, with their stepwise training requiring more than 5 h for the inspection of 25 models.

7. Conclusions

A step towards a systematic investigation of the applicability of supervised classification of spectrograms through a 2D-CNN and an unsupervised abnormality detection through an LSTM-AE methodology for leakage detection in oil pipelines was described. An edge computing system placed alongside the pipelines that can acquire data from accelerometers mounted on the pipes, process them using the described LSTM-AE methodology and issue early warning notifications on leakages was used. Whilst single-axis sensors were considered here,

Table 19

Performance comparison between the proposed networks and benchmark models across 5 metrics.

Model	Accuracy (%)	Precision (%)	Recall (%)	Specificity (%)	F1 (%)
CNN classifier	92	92	90	92	94
LSTM AE	90	90	90	92	94
OC SVM	72	58	80	62	72
Isolation Forest	72	58	86	60	70
ARMA and z-score	88	86	86	90	90

cases of 3-axes sensors can be examined by directly expanding the described approach. Several values of leakage diameter and distance from the accelerometer were applied on a scaled experimental apparatus, filled with dry spherical particles, to verify the accuracy of the methodologies. Besides, the efficacy of our approach was tested in a real-world scenario in an oil refinery premises in Greece. Special attention was paid to the comparison of the results against other popular anomaly detection algorithms that are widely employed in the pertinent literature.

One main challenge that our work focused upon was the timely detection of malfunctions from sensors integrated into pipelines set in industrial and harsh environments. It was manifested that, although the LSTM AE model is exact in identifying anomalies that signify malfunction of the piping system, it is susceptible to false negatives. It was explicitly demonstrated that the signal from the pipeline wall, namely the signal before the inducement of the artificial leakage, was of the same order of magnitude as the noise produced by the pipe wall for small leakages, rendering the leakage detection effectively indiscernible. Conversely, the 2D-CNN models were employed in classifying spectrograms in which the time point when the leakage occurred was purposefully omitted. This approach offers an alternative to storing lengthy signals by storing static low-resolution images that occupy considerably less memory instead, while effectively identifying very small leakages. Furthermore, our experimentation with multiple colormaps for the generation of the spectrograms indicated that greater performance was achieved by using spectrograms with one predominant color component in the RGB color panel (i.e. in our study case Blue was used). Besides the results demonstrated that our two proposed deep learning models outperformed every benchmark model, being overall more accurate. Additionally, it was also found that the neural networks were significantly more efficient to tune than the ARMA models, exploiting the computational capabilities of state-of-the-art GPUs.

While accuracy is of utmost importance, the selection of the proposed methodologies was driven by two additional requirements: (a) the need for minimal supervision, which is critical since training on real-case malfunctions is difficult and (b) the need for efficiency when triggering events are not clear, since distributed edge computing nodes normally operate in a modular manner (i.e. period for acquisition, period for processing, period for sleep, etc.). The main benefit of the LSTM-AE method proved to be efficient for the former requirement while the main advantage of the 2D-CNN proved to be its ability to perform well for the latter. As a result, the two methods could be part of a robust early warning system that can contribute to preventing possible environmental disasters and incidents in the fuel industry and the future evolution of intelligent sensor solutions for liquid and gas storing and transporting procedures. Combination of these two methods in the same system (e.g. in sequential form) or different layers of the same hardware architecture (e.g. LSTM-AE on microcontroller and 2D-CNN on microprocessor level) could be beneficial in terms of accuracy, resource efficiency, and is currently under development.

CRediT authorship contribution statement

Christos Spandonidis: Conceptualization, Methodology, Data curation, Visualization, Investigation, Supervision, Validation, Writing – reviewing. **Panayiotis Theodoropoulos:** Methodology, Software, Data curation, Writing – original draft, Visualization, Investigation, Validation. **Fotis Giannopoulos:** Data Curation, Writing – review & editing. **Nektarios Galiatsatos:** Data Curation. **Areti Petsa:** Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research has been co-financed by the European Regional Development Fund of the European Union and Greek national funds through the Operational Program Competitiveness, Entrepreneurship, and Innovation, under the call RESEARCH – CREATE – INNOVATE (project code: T1EDK-00791).

References

- Abdeljaber, O., Avci, O., Kiranyaz, M.S., Boashash, B., Sodano, H., Inman, D.J., 2018. 1-D CNNs for structural damage detection: Verification on a structural health monitoring benchmark data. *Neurocomputing* 275, 1308–1317. <http://dx.doi.org/10.1016/j.neucom.2017.09.069>.
- Abdeljaber, O., Avci, O., Kiranyaz, S., Gabbouj, M., Inman, D.J., 2017. Real-time vibration-based structural damage detection using one-dimensional convolutional neural networks. *J. Sound Vib.* 388, 154–170. <http://dx.doi.org/10.1016/j.jsv.2016.10.043>.
- Adegboye, M.A., Fung, W.-K., Karnik, A., 2019. Recent advances in pipeline monitoring and oil leakage detection technologies: Principles and approaches. *Sensors* 19, 2548. <http://dx.doi.org/10.3390/s19112548>.
- Altabay, W.A., Noori, M., Wang, T., Ghiasi, R., Kuok, S.-C., Wu, Z., 2021. Deep learning-based crack identification for steel pipelines by extracting features from 3D shadow modeling. *Appl. Sci.* 11, 6063. <http://dx.doi.org/10.3390/app11136063>.
- Amini, I., Jing, Y., Chen, T., Colin, A., Meyer, G., 2020. A two-stage deep-learning based detection method for pipeline leakage and transient conditions. In: 2020 IEEE Electric Power and Energy Conference (EPEC). IEEE, pp. 1–5. <http://dx.doi.org/10.1109/EPEC48502.2020.9320021>.
- Aryai, V., Baji, H., Mahmoodian, M., 2022. Failure assessment of corrosion affected pipeline networks with limited failure data availability. *Process Saf. Environ. Prot.* 157, 306–319. <http://dx.doi.org/10.1016/j.psep.2021.11.024>.
- Basora, L., Olive, X., Dubot, T., 2019. Recent advances in anomaly detection methods applied to aviation. *Aerospace* 6, 117. <http://dx.doi.org/10.3390/aerospace6110117>.
- Bianchi, F.M., Espeseth, M.M., Borch, N., 2020. Large-scale detection and categorization of oil spills from SAR images with deep learning. *Remote Sens.* 12, 2260. <http://dx.doi.org/10.3390/rs12142260>.
- Braei, M., Wagner, S., 2020. Anomaly detection in univariate time-series: A survey on the state-of-the-art [WWW Document]. <http://dx.doi.org/10.48550/arXiv.2004.00433>.
- Callan, T., 2008. Pipeline technology today and tomorrow. *OIL GAS Eur. Mag.* 110–115.
- Chen, J., Wu, H., Liu, X., Xiao, Y., Wang, M., Yang, M., Rao, Y., 2018. A real-time distributed deep learning approach for intelligent event recognition in long distance pipeline monitoring with DOFS. In: 2018 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC). IEEE, pp. 290–2906. <http://dx.doi.org/10.1109/CyberC.2018.00059>.
- Christos, S.C., Fotis, G., Nektarios, G., Dimitris, R., Areti, P., Dimitrios, S., 2021a. Autonomous low-cost wireless sensor platform for leakage detection in oil and gas pipes. In: 2021 10th International Conference on Modern Circuits and Systems Technologies (MOCAST). IEEE, pp. 1–4. <http://dx.doi.org/10.1109/MOCAST52088.2021.9493340>.
- Christos, S.C., Nektarios, G., Fotios, G., Nikolaos, D., Panagiotis, P., Areti, P., 2021b. Development of an IoT early warning platform for augmented decision support in oil & gas. In: 2021 10th International Conference on Modern Circuits and Systems Technologies (MOCAST). IEEE, pp. 1–4. <http://dx.doi.org/10.1109/MOCAST52088.2021.9493340>.
- Christos, S.C., Panagiotis, T., Christos, G., 2020. Combined multi-layered big data and responsible AI techniques for enhanced decision support in Shipping. In: 2020 International Conference on Decision Aid Sciences and Application (DASA). IEEE, pp. 669–673. <http://dx.doi.org/10.1109/DASA51403.2020.9317030>.

- Cody, R.A., Tolson, B.A., Orchard, J., 2020. Detecting leaks in water distribution pipes using a deep autoencoder and hydroacoustic spectrograms. *J. Comput. Civ. Eng.* 34, 04020001. [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0000881](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0000881).
- Gargade, A., Ohol, S., 2021. Design and development of in-pipe inspection robot for various pipe sizes. *IOP Conf. Ser. Mater. Sci. Eng.* 1012, 012001. <http://dx.doi.org/10.1088/1757-899X/1012/1/012001>.
- Hoarau, Q., Ginolhac, G., Atto, A.M., Nicolas, J.M., 2017. Robust adaptive detection of buried pipes using GPR. *Signal Process.* 132, 293–305. <http://dx.doi.org/10.1016/j.sigpro.2016.07.001>.
- Hu, Z., Chen, B., Chen, W., Tan, D., Shen, D., 2021a. Review of model-based and data-driven approaches for leak detection and location in water distribution systems. *Water Supply* 21, 3282–3306. <http://dx.doi.org/10.2166/ws.2021.101>.
- Hu, X., Han, Y., Yu, B., Geng, Z., Fan, J., 2021b. Novel leakage detection and water loss management of urban water supply network using multiscale neural networks. *J. Clean. Prod.* 278, 123611. <http://dx.doi.org/10.1016/j.jclepro.2020.123611>.
- Jia, Z., Ren, L., Li, H., Sun, W., 2018. Pipeline leak localization based on FBG hoop strain sensors combined with BP neural network. *Appl. Sci.* 8 (146), <http://dx.doi.org/10.3390/app8020146>.
- Kang, J., Park, Y.-J., Lee, J., Wang, S.-H., Eom, D.-S., 2018. Novel leakage detection by ensemble CNN-SVM and graph-based localization in water distribution systems. *IEEE Trans. Ind. Electron.* 65, 4279–4289. <http://dx.doi.org/10.1109/TIE.2017.2764861>.
- Khan, F., Yarveisy, R., Abbassi, R., 2021. Risk-based pipeline integrity management: A road map for the resilient pipelines. *J. Pipeline Sci. Eng.* 1, 74–87. <http://dx.doi.org/10.1016/j.jpse.2021.02.001>.
- Li, Y.T., Li, X., Cai, G.W., Yang, L.H., 2013. Influence of AC interference to corrosion of Q235 carbon steel. *Corros. Eng. Sci. Technol.* 48, 322–326. <http://dx.doi.org/10.1179/1743278212Y.0000000076>.
- Li, J., Liu, Y., Chai, Y., He, H., Gao, M., 2019. A small leakage detection approach for gas pipelines based on CNN. In: 2019 CAA Symposium on Fault Detection, Supervision and Safety for Technical Processes (SAFEPROCESS). IEEE, pp. 390–394. <http://dx.doi.org/10.1109/SAFEPROCESS45799.2019.9213371>.
- Liao, Z., Yan, H., Tang, Z., Chu, X., Tao, T., 2021. Deep learning identifies leak in water pipeline system using transient frequency response. *Process Saf. Environ. Prot.* 155, 355–365. <http://dx.doi.org/10.1016/j.psep.2021.09.033>.
- Lin, Y., Nie, Z., Ma, H., 2017. Structural damage detection with automatic feature-extraction through deep learning. *Comput. Civ. Infrastruct. Eng.* 32, 1025–1046. <http://dx.doi.org/10.1111/mice.12313>.
- Liu, H., pipeline. Britannica.
- Lu, J., Huang, J., Lu, F., 2020. Kernel extreme learning machine with iterative picking scheme for failure diagnosis of a turbofan engine. *Aerosp. Sci. Technol.* 96, 105539. <http://dx.doi.org/10.1016/j.ast.2019.105539>.
- Lu, C., Wang, Z., Zhou, B., 2017. Intelligent fault diagnosis of rolling bearing using hierarchical convolutional network based health state classification. *Adv. Eng. Inform.* 32, 139–151. <http://dx.doi.org/10.1016/j.aei.2017.02.005>.
- MacDonald, C., Yang, M., Learn, S., Hugo, R., Park, S., 2020. Liquid pipeline rupture detection using multiple artificial intelligence classifiers during steady-state and transient operations. In: Volume 3: Operations, Monitoring, and Maintenance; Materials and Joining. American Society of Mechanical Engineers, <http://dx.doi.org/10.1115/IPC2020-9237>.
- Mahmutoglu, Y., Turk, K., 2018. A passive acoustic based system to locate leak hole in underwater natural gas pipelines. *Digit. Signal Process.* 76, 59–65. <http://dx.doi.org/10.1016/j.dsp.2018.02.007>.
- Manekiya, M.H., Arulmozhivarman, P., 2016. Leakage detection and estimation using IR thermography. In: 2016 International Conference on Communication and Signal Processing (ICCSP). IEEE, pp. 1516–1519. <http://dx.doi.org/10.1109/ICCSP.2016.7754411>.
- Martini, A., Troncosi, M., Rivola, A., 2016. Leak detection in water-filled small-diameter polyethylene pipes by means of acoustic emission measurements. *Appl. Sci.* 7 (2), <http://dx.doi.org/10.3390/app7010002>.
- Mpesha, W., Hanif Chaudhry, M., Gassman, S.L., 2002. Leak detection in pipes by frequency response method using a step excitation. *J. Hydraul. Res.* 40, 55–62. <http://dx.doi.org/10.1080/00221680209499873>.
- Ni, S.-H., Huang, Y.-H., Lo, K.-F., Lin, D.-C., 2010. Buried pipe detection by ground penetrating radar using the discrete wavelet transform. *Comput. Geotech.* 37, 440–448. <http://dx.doi.org/10.1016/j.compgeo.2010.01.003>.
- Nikolaïdis, S., Porlidas, D., Glentis, G.-O., Kalfas, A., Spandonidis, C., 2019. Smart sensor system for leakage detection in pipes carrying oil products in noisy environment: The ESTHISIS project. In: 2019 29th International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS). pp. 125–126. <http://dx.doi.org/10.1109/PATMOS.2019.8862111>.
- Png, W.H., Lin, H.S., Pua, C.H., Rahman, F.A., 2018. Pipeline monitoring and leak detection using loop integrated Mach Zehnder Interferometer optical fiber sensor. *Opt. Fiber Technol., Mater. Devices Syst.* 46, 221–225. <http://dx.doi.org/10.1016/j.yofte.2018.10.013>.
- Qin, Y., Lou, Y., 2019. Hydrological time series anomaly pattern detection based on isolation forest. In: 2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC). IEEE, pp. 1706–1710. <http://dx.doi.org/10.1109/ITNEC.2019.8729405>.
- Rehman, K., Nawaz, F., 2017. Remote pipeline monitoring using wireless sensor networks. In: 2017 International Conference on Communication, Computing and Digital Systems (C-CODE). IEEE, pp. 32–37. <http://dx.doi.org/10.1109/C-CODE.2017.7918897>.
- Shi, Y., Wang, Y., Zhao, L., Fan, Z., 2019. An event recognition method for Φ -OTDR sensing system based on deep learning. *Sensors* 19 (3421), <http://dx.doi.org/10.3390/s19153421>.
- Shibata, A., Konishi, M., Abe, Y., Hasegawa, R., Watanabe, M., Kamijo, H., 2009. Neuro based classification of gas leakage sounds in pipeline. In: 2009 International Conference on Networking, Sensing and Control. IEEE, pp. 298–302. <http://dx.doi.org/10.1109/ICNSC.2009.4919290>.
- Shravani, D., Prajwal, Y.R., Prapulla, S.B., Salanke, N.S.G.R., Shobha, G., Ahmad, S.F., 2019. A machine learning approach to water leak localization. In: 2019 4th International Conference on Computational Systems and Information Technology for Sustainable Solution (CSITSS). IEEE, pp. 1–6. <http://dx.doi.org/10.1109/CSITSS47250.2019.9031010>.
- Shukla, H., Piratla, K., 2020. Leakage detection in water pipelines using supervised classification of acceleration signals. *Autom. Constr.* 117, 103256. <http://dx.doi.org/10.1016/j.autcon.2020.103256>.
- Siswanto, N., Doğan, A., Priyanta, D., Zaman, M.B., Semin, S., 2019. Possibility of piezoelectric sensor to monitor onshore pipeline in real time monitoring. *Int. J. Mar. Eng. Innov. Res.* 3, <http://dx.doi.org/10.12962/j25481479.v3i4.4951>.
- Theodoropoulos, P., Spandonidis, C.C., Fassois, S., 2022. Use of convolutional neural networks for vessel performance optimization and safety enhancement. *Ocean Eng.* 248, 110771. <http://dx.doi.org/10.1016/j.oceaneng.2022.110771>.
- Theodoropoulos, P., Spandonidis, C.C., Giannopoulos, F., Fassois, S., 2021. A deep learning-based fault detection model for optimization of shipping operations and enhancement of maritime safety. *Sensors* 21 (16), 5658. <http://dx.doi.org/10.3390/s21165658>.
- Wang, C., Han, F., Zhang, Y., Lu, J., 2020. An SAE-based resampling SVM ensemble learning paradigm for pipeline leakage detection. *Neurocomputing* 403, 237–246. <http://dx.doi.org/10.1016/j.neucom.2020.04.105>.
- Wang, W., Zhang, Y., Li, Y., Hu, Q., Liu, Chengsong, Liu, Cuiwei, 2022. Vulnerability analysis method based on risk assessment for gas transmission capabilities of natural gas pipeline networks. *Reliab. Eng. Syst. Saf.* 218, 108150. <http://dx.doi.org/10.1016/j.res.2021.108150>.
- Weimer, D., Scholz-Reiter, B., Shpitalni, M., 2016. Design of deep convolutional neural network architectures for automated feature extraction in industrial inspection. *CIRP Ann.* 65, 417–420. <http://dx.doi.org/10.1016/j.cirp.2016.04.072>.
- Xie, M., Tian, Z., 2018. A review on pipeline integrity management utilizing in-line inspection data. *Eng. Fail. Anal.* 92, 222–239. <http://dx.doi.org/10.1016/j.engfailanal.2018.05.010>.
- Xie, Y., Xiao, Y., Liu, X., Liu, G., Jiang, W., Qin, J., 2020. Time-frequency distribution map-based convolutional neural network (CNN) model for underwater pipeline leakage detection using acoustic signals. *Sensors* 20 (5040), <http://dx.doi.org/10.3390/s20185040>.
- Yazdekhesti, S., Piratla, K.R., Atamturktur, S., Khan, A.A., 2017. Novel vibration-based technique for detecting water pipeline leakage. *Struct. Infrastruct. Eng.* 13, 731–742. <http://dx.doi.org/10.1080/15732479.2016.1188318>.
- Zhang, J., Cosma, G., Watkins, J., 2021. Image enhanced mask R-CNN: A deep learning pipeline with new evaluation measures for wind turbine blade defect detection and classification. *J. Imaging* 7, 46. <http://dx.doi.org/10.3390/jimaging7030046>.
- Zhang, R., Shaoyan, Z., Lan, Y., Jianmin, J., 2008. Network anomaly detection using one class support vector machine. In: International MultiConference of Engineers and Computer Scientists.
- Zhang, R., Zhang, S., Muthuraman, S., Jiang, J., 2007. One class support vector machine for anomaly detection in the communication network performance data. In: ELEC-TROSCIENCE'07: Proceedings of the 5th Conference on Applied Electromagnetics, Wireless and Optical Communications. pp. 31–37.
- Zhong, S., Fu, S., Lin, L., Fu, X., Cui, Z., Wang, R., 2019. A novel unsupervised anomaly detection for gas turbine using isolation forest. In: 2019 IEEE International Conference on Prognostics and Health Management (ICPHM). IEEE, pp. 1–6. <http://dx.doi.org/10.1109/ICPHM.2019.8819409>.
- Zhou, M., Yang, Y., Xu, Y., Hu, Y., Cai, Y., Lin, J., Pan, H., 2021. A pipeline leak detection and localization approach based on ensemble TL1DCNN. *IEEE Access* 9, 47565–47578. <http://dx.doi.org/10.1109/ACCESS.2021.3068292>.